

Contents

MP Land Intelligence Platform	1
Engineering PRD — v1.0	1

MP Land Intelligence Platform

Engineering PRD — v1.0

For: 2 developers **Duration:** 16 weeks to production v1 **Scope:** Madhya Pradesh, Indore district first **Status:** Build spec. Everything here is buildable. Anything not here is out of scope.

0. How to use this document

Read sections 1–3 first, together, in one sitting. Then Dev A owns sections 5–7, Dev B owns sections 8–9. Both own section 10.

Rules for this project:

If something is not in this document, it is not in v1. Raise it, do not build it. Section 10 (Precautions) overrides every other section. If a precaution conflicts with a feature, the precaution wins. Section 13 is the only definition of “done.”

1. Objective

Build a system that lets a small investment team find, evaluate and monitor land parcels in Madhya Pradesh using public data, well enough to make real acquisition decisions.

Success in 16 weeks means all of the following are true:

Every khasra in Indore district is in the database, with ownership and transaction history. A team member can open a map, turn on six layers, zoom to a parcel, and see a complete dossier. A team member can run any of 30 pre-defined screens with adjustable parameters and get a ranked list. Every live bank auction in MP is automatically scored against our own valuation, within 24 hours of listing. We have used the system to evaluate at least 20 real opportunities and bid on at least 3.

Not success: a beautiful platform nobody has made a decision with.

2. Scope

In scope (v1)

Area	v1 scope
Geography	Indore district at full depth; all 55 MP districts at ownership + deed depth
Sources	12 sources listed in section 5. No others.
Map	MapLibre, 6 core layers, zoom to parcel, click for dossier
Screening	30 parameterised queries from the playbook
Entity resolution	Deterministic + fuzzy matching, human review queue. No ML model.
Alerts	Watchlist on parcels and entities, email digest
Users	3–6 internal users. Simple auth. No multi-tenancy, no billing, no roles beyond admin/user.

Additionally in scope (v1):

- Natural language query interface
- Political/official connectivity graph
- Graph database (Neo4j etc.) — Postgres recursive CTEs are sufficient at this scale
- Feature store, MLflow, Spark

Explicitly out of scope (v1) Do not build these. They are v2 or later.

- Predictive ML models
- Mobile app
- PDF report generator
- Public/customer-facing anything
- Real-time streaming (Kafka etc.)
- Satellite change detection beyond nightlight + built-up index

Why these are cut Two developers. Every one of the above adds a week minimum and none of them changes whether you can make an investment decision in month 4.

3. Team split

Dev A — Data

Owns: scrapers, ingestion pipelines, parsing, entity resolution, corpus schema, data quality, scheduling.

Primary skills needed: Python, Postgres, HTTP/scraping, some geospatial.

Dev B — Platform

Owns: API, vector tiles, map frontend, dossier UI, screening UI, alerts, deployment, monitoring.

Primary skills needed: Python (FastAPI), TypeScript, MapLibre, PostGIS querying.

Shared

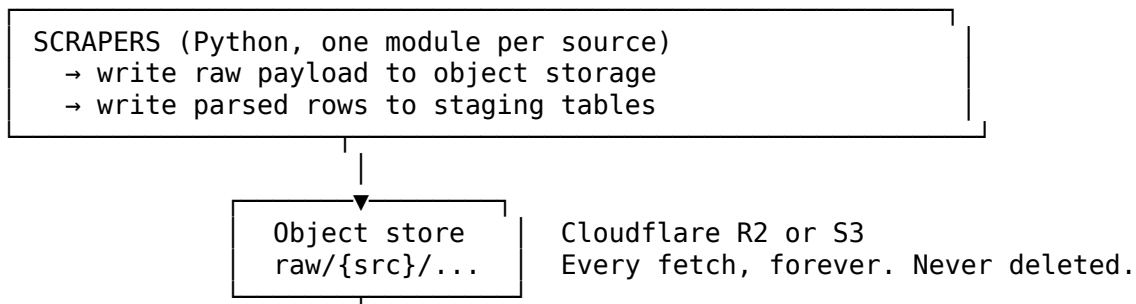
Schema design — decided together in week 1, changed only by agreement Section 10 precautions — both enforce Weekly demo to the founders, every Friday, using real data

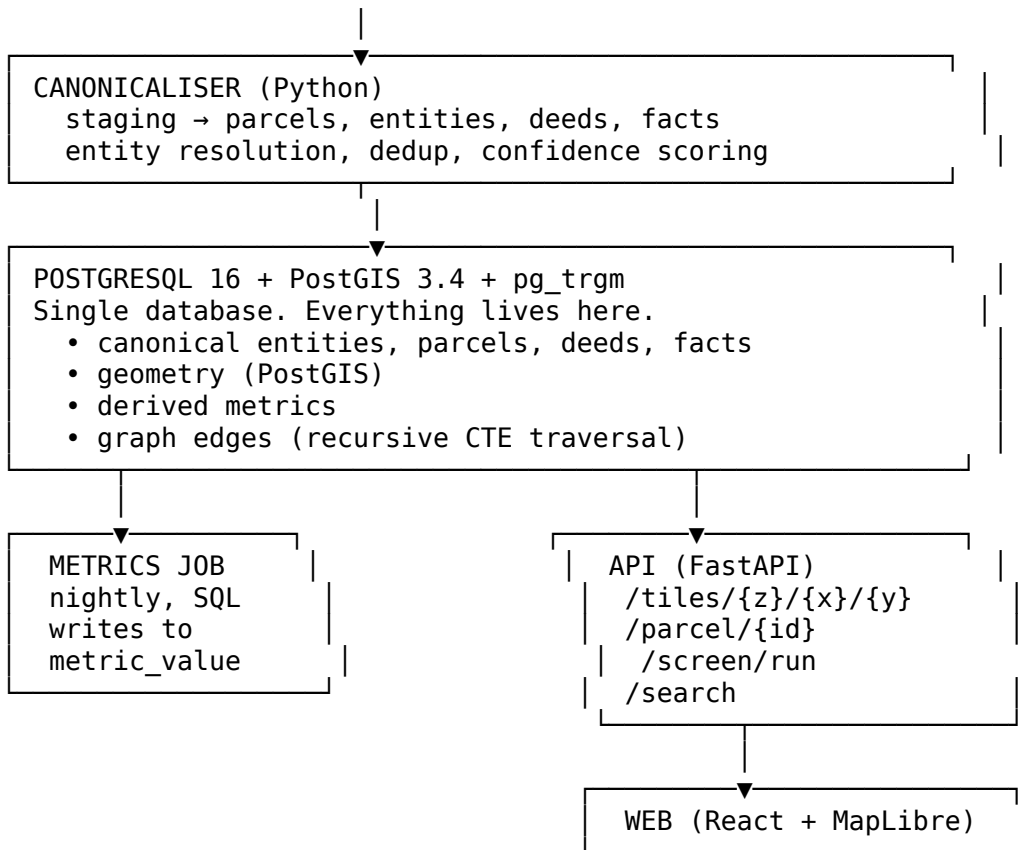
Ways of working

One repo, monorepo layout: /ingest, /api, /web, /db, /ops Every scraper has a fixture test with a saved HTML/JSON sample Schema changes via numbered migration files only (db/migrations/0007_add_x.sql). No manual DDL on the server, ever. Deploy: single VM to start. Do not build Kubernetes.

4. Architecture

Deliberately boring. One database, one API, one frontend, one scheduler.





SCHEDULER: systemd timers or cron. Not Airflow. Not Prefect.
If you outgrow cron, revisit at v2.

Stack decisions, fixed

Layer	Choice	Why not the alternative
DB	PostgreSQL 16 + PostGIS	One database is two developers' entire operational capacity
Raw storage	Cloudflare R2	S3-compatible, no egress fees, cheaper for this
Scraping	httpx + selectolax; playwright only where JS is mandatory	Scrapy is overkill for ~12 sources
OCR/extraction	Claude API for documents; pdfplumber for structured PDFs	Do not build a Tesseract pipeline
Scheduling	systemd timers	Airflow costs a week of setup and needs babysitting
API	FastAPI	—
Tiles	ST_AsMVT served from Postgres, cached in R2 + Cloudflare CDN	Tippecanoe pre-generation is a v2 optimisation
Frontend	React + Vite + MapLibre GL JS + TanStack Query	—
Auth	Single-tenant, email allowlist + magic link	Do not integrate an identity provider

Layer	Choice	Why not the alternative
Hosting	One 8-core / 32GB VM + managed Postgres	Kubernetes is not a v1 problem

5. Data sources — scraper specifications

Twelve sources. Numbered by build priority. Dev A builds in this order.

Day 1 task before writing any scraper: manually walk each portal, save one sample response per document type into `/ingest/fixtures/`, and record the actual request shape. The specs below describe intent and constraints; the exact parameter names must be confirmed from the live portal, not assumed.

S1 — MP Bhulekh (mpbhulekh.gov.in) — CRITICAL Gives: khasra records, khatauni, B1 (loans/charges), owner names, area, land classification, tenure, cultivator entries, ULPIN.

Key advantage: MP exposes **ULPIN** (Unique Land Parcel Identification Number). Use ULPIN as the primary parcel key wherever present. This solves most of the parcel-identity problem that makes other states painful.

Property	Spec
Path	Free Services (no-login public path)
Hierarchy	District → Tehsil → Village → Khasra
Auth	None for free services. Do not use the login/paid path.
Captcha	Expect one. Budget for 2Captcha integration.
Rate limit	Max 1 request per 3 seconds per IP.
Concurrency	Non-negotiable.
Cadence	Max 3 workers, each on a different residential IP
Volume	Indore: monthly full sweep. Other districts: quarterly.
Output	Indore ~1.5-2M khasra. At 1 req/3s across 3 workers ≈ 20 days for a full sweep. Plan for it. <code>stg_bhulekh_khasra</code>

Precautions:

Enumerate village-by-village, never by random khasra guessing. Persist a cursor (`district/tehsil/village/`) so a crash resumes, not restarts. If you get a block, stop for 24 hours. Do not rotate through IPs to defeat it. Save every raw response to R2 before parsing.

S2 — SAMPADA 2.0 / IGRS MP — CRITICAL Gives: registered deeds — sale, gift, mortgage, lease, power of attorney, agreement to sell, partition. Parties, consideration value, stamp duty, property description, date.

This is the highest-value source in the system. The POA and agreement-to-sell documents are the pre-transaction signals nobody else scrapes.

Property	Spec
Scope	Scrape the full document index, not just sale deeds.

Property	Spec
Hierarchy	SRO office → date range → document list → document detail
Auth	Public search where available; confirm on day 1
Page types	Two kinds of pages exist: no-login (public search, less info per document) and logged-in (fuller data on the same document). Confirm the exact field differential between the two on the day 1 walkthrough.
Rate limit	1 req / 2s per IP
Cadence	Indore: daily. Other districts: weekly.
Backfill	2015 onward for Indore; 2019 onward statewide
Output	stg_sampada_doc, stg_sampada_party

Precautions:

Property descriptions are free text. Parse khasra/village with a dedicated matcher, log every failure to review_queue rather than dropping it. Consideration value is systematically understated. Never present it as market value anywhere in the UI.

S3 — MP Bhu-Naksha / WebGIS 2.0 — HIGH Gives: cadastral parcel geometry.

Property	Spec
Output	Parcel polygons keyed by village + khasra (and ULPIN where present)
Coverage	Partial. Assume 40–70% in Indore, lower rurally.
Cadence	One-time bulk pull, then quarterly refresh for changed villages
Format	Convert everything to EPSG:4326. Validate with ST_IsValid, repair with ST_MakeValid.

Precautions — this is the one most likely to cause real-world harm:

Set boundary_tier on every parcel: A = real digitised polygon, B = village polygon only, C = no geometry. **Tier B and C parcels must render visually differently on the map (dashed outline, hatched fill).** Never draw an approximated boundary as if it were surveyed. Sanity-check every polygon against the MP bounding box. Reject anything outside it.

S4 — MP RERA (rera.mp.gov.in) — HIGH Gives: registered projects, promoter identity, declared completion dates, unit counts, quarterly progress, extensions.

Property	Spec
Auth	None
Rate limit	1 req / 2s
Cadence	Weekly
Volume	Low — a few thousand projects statewide
Output	stg_rera_project, stg_rera_update

S5 — Auction feeds — HIGH Three sub-sources into one table.

Sub-source	What
IBAPI (ibapi.in)	Bank auctions under SARFAESI
MP e-auction portal	State/authority auctions
DRT auction notices	Debt recovery tribunal

Property	Spec
Cadence	Every 6 hours. Auction windows are short; this is where speed is worth money.
Filter	Madhya Pradesh only
Output	stg_auction
Alert	New MP land listing → immediate scoring → email within 1 hour

Precautions:

Extract and store the possession type verbatim (symbolic vs physical). Surface it prominently. This single field is where most auction buyers lose money.

S6 — MCA21 — MEDIUM Gives: company master data, directors, charges (CHG-1 filings), registered addresses, status.

Property	Spec
Portals	Master data collection has two distinct website types: i) BPC — login required, direct download. ii) All Pass — no login required, but no direct download; requires browser automation (Selenium) to extract.
Scope v1	Only companies that appear as a party in a deed, or as a RERA promoter. Do not bulk-scrape MCA.
Cadence	On-demand when a new company appears, then monthly refresh for tracked companies
Output	stg_mca_company, stg_mca_director, stg_mca_charge

Why charges matter: a new charge registration against land is a 6-18 month leading indicator that a project is about to start. This is one of the two highest-value early signals in the whole system.

S7 — Master plans & zoning (MP Town & Country Planning) — MEDIUM Gives: land use zones, permissible FSI, road network, reservations, no-development zones.

Property	Spec
Scope v1	Indore and Bhopal only.
Method	Mostly manual. Download plan PDFs and any shapefiles; georeference zone maps in QGIS against satellite base imagery.

Property	Spec
Effort	Budget 2 weeks of one developer's time for Indore. This is not automatable.
Output	zone table with polygons + FSI attributes

Precautions:

Store the georeferencing RMS error per plan sheet. Anything above ~15m at parcel scale is unusable for boundary-adjacent decisions — flag it. Extract only what you need: zone polygon, zone code, permissible FSI, road width table, reservation polygons. Ignore the other 280 pages.

S8 — Satellite: Sentinel-2 + VIIRS — MEDIUM

Property	Spec
Sentinel-2	AWS Open Data S3 via STAC. Quarterly dry-season cloud-free composites.
Derived	NDVI, NDBI (built-up index), NDWI (water) at 10m, aggregated to H3 res-8 cells
VIIRS	Monthly nightlight composites from NOAA/EOG. Aggregate to H3 res-7.
Scope v1	Indore district only for Sentinel. VIIRS statewide (it's cheap).
Cadence	Quarterly (Sentinel), monthly (VIIRS)
Output	metric_value rows keyed to H3 cells

Precautions:

Do **not** build a per-parcel satellite pipeline in v1. Cell-level aggregates only. Per-parcel analysis is a v2 feature and needs sub-metre imagery you are not buying yet. Monsoon imagery is unusable. Use Nov-Mar composites only.

S9 — MP e-Procurement (mptenders.gov.in) — MEDIUM Gives: infrastructure tenders — NIT, award, work order status.

Property	Spec
Filter	Roads, bridges, water supply, sewerage, metro, urban development
Cadence	Weekly
Spatial	Extract location text → geocode → store as point or corridor buffer. Log unresolvable ones.
Output	stg_tender, infra_project

S10 — Circle rates — LOW effort, HIGH value Gives: government benchmark valuation by location.

Property	Spec
Source	MP registration department published rate tables (via SAMPADA / IGRS)
Cadence	Annually, plus on any revision notification

Property	Spec
Format Output	Usually PDF. Parse to a location → rate table. circle_rate

Essential — half the market metrics normalise against this.

S11 — RCMS (revenue court cases) — LOW Gives: revenue litigation on parcels — partition, mutation disputes, boundary cases.

Property	Spec
Scope v1	Indore only, on-demand lookup during parcel diligence
Cadence	On-demand, not bulk
Output	stg_rcms_case

Precaution: absence of a case is weak evidence. Never display “no litigation” — display “no case found in RCMS as of {date}”.

S12 — ECI (Election Commission of India) — LOW Gives: candidate election affidavits — declared assets, liabilities, criminal cases, education, and family details for contesting candidates, publicly filed under the Representation of the People Act.

Property	Spec
Path	Public affidavit search (no login)
Hierarchy	State → constituency → election year → candidate
Auth	None
Rate limit	1 req / 2s per IP
Cadence	On-demand, refreshed after each election cycle
Output	stg_eci_candidate, stg_eci_affidavit

Precautions:

This is the raw material for the political/official connectivity graph — treat every match to a private landholding entity as a lead for the review queue, not an automatic link. Names in affidavits are self-reported; run them through the same entity resolution pipeline as every other source, do not special-case them.

(Note: rate limit, cadence, and output table names for S12 are drafted to match the format of S1-S11 and should be confirmed against the live ECI portal on the day 1 walkthrough, same as every other source.)

6. Database schema

Full DDL. This is the contract between Dev A and Dev B. Changes require both to agree.

```
-- Every single fetch from every source. Never deleted.
CREATE TABLE src_fetch (
  id          BIGSERIAL PRIMARY KEY,
  source      TEXT NOT NULL,      -- 'bhulekh', 'sampada', 'rera', ...
  doc_type    TEXT NOT NULL,      -- 'khasra', 'sale_deed', 'project', ...
```

```

url          TEXT,
request      JSONB,
fetched_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
http_status  INT,
raw_key      TEXT NOT NULL,
content_hash TEXT NOT NULL,
parser_version TEXT NOT NULL,
parse_status TEXT NOT NULL DEFAULT 'pending' -- pending|ok|failed|partial
);
CREATE INDEX ON src_fetch (source, doc_type, fetched_at DESC);
CREATE INDEX ON src_fetch (content_hash);

```

6.1 Provenance

```

CREATE TABLE admin_unit (
  id          BIGSERIAL PRIMARY KEY,
  level       TEXT NOT NULL CHECK (level IN ('state','district','tehsil','village','ulb','wa
  name        TEXT NOT NULL,
  name_hi     TEXT,
  lgd_code    TEXT,
  parent_id   BIGINT REFERENCES admin_unit(id),
  geom        geometry(MultiPolygon, 4326),
  UNIQUE (level, name, parent_id)
);
CREATE INDEX ON admin_unit USING GIST (geom);
CREATE INDEX ON admin_unit (parent_id, level);

```

6.2 Administrative geography

```

CREATE TABLE parcel (
  id          BIGSERIAL PRIMARY KEY,
  ulpin       TEXT UNIQUE,
  village_id  BIGINT NOT NULL REFERENCES admin_unit(id),
  khasra_no   TEXT NOT NULL,
  area_recorded NUMERIC(12,4),
  area_geom   NUMERIC(12,4),
  land_class  TEXT,
  tenure_class TEXT,
  geom        geometry(MultiPolygon, 4326),
  centroid    geometry(Point, 4326),
  h3_r8       TEXT,
  boundary_tier CHAR(1) NOT NULL DEFAULT 'C' CHECK (boundary_tier IN ('A','B','C')),
  first_seen  TIMESTAMPTZ NOT NULL DEFAULT now(),
  last_seen   TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (village_id, khasra_no)
);
CREATE INDEX ON parcel USING GIST (geom);
CREATE INDEX ON parcel USING GIST (centroid);
CREATE INDEX ON parcel (h3_r8);
CREATE INDEX ON parcel (village_id);

```

6.3 Parcels boundary_tier: **A** = digitised polygon from Bhu-Naksha. **B** = village polygon only, position approximate. **C** = no geometry. Dev B must style these differently on the map.

```

CREATE TABLE entity (
  id          BIGSERIAL PRIMARY KEY,
  kind        TEXT NOT NULL CHECK (kind IN ('person','company','trust','govt','unknown')),
  canonical_name TEXT NOT NULL,
  father_name  TEXT,                                -- key matching field in Indian records
  address_text TEXT,
  village_id   BIGINT REFERENCES admin_unit(id),
  cin          TEXT,                                -- companies only
  cluster_id   BIGINT,                              -- FK added after entity_cluster
  created_at   TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX ON entity USING GIN (canonical_name gin_trgm_ops);
CREATE INDEX ON entity (cin) WHERE cin IS NOT NULL;
CREATE INDEX ON entity (cluster_id);

-- Every name variant ever seen for this entity, with where it came from.
CREATE TABLE entity_alias (
  id          BIGSERIAL PRIMARY KEY,
  entity_id   BIGINT NOT NULL REFERENCES entity(id),
  alias       TEXT NOT NULL,
  source      TEXT NOT NULL,
  fetch_id    BIGINT REFERENCES src_fetch(id)
);
CREATE INDEX ON entity_alias USING GIN (alias gin_trgm_ops);

-- Resolved groups. Merges are reversible.
CREATE TABLE entity_cluster (
  id          BIGSERIAL PRIMARY KEY,
  label       TEXT,
  confidence   NUMERIC(3,2) NOT NULL,             -- lowest-confidence merge in the cluster
  method       TEXT NOT NULL,                     -- 'exact_cin'|'fuzzy'|'manual'
  reviewed_by  TEXT,
  created_at   TIMESTAMPTZ NOT NULL DEFAULT now()
);
ALTER TABLE entity ADD CONSTRAINT fk_cluster
  FOREIGN KEY (cluster_id) REFERENCES entity_cluster(id);

-- Graph edges. Traverse with recursive CTEs; no graph DB needed at this scale.
CREATE TABLE entity_link (
  id          BIGSERIAL PRIMARY KEY,
  from_id     BIGINT NOT NULL REFERENCES entity(id),
  to_id       BIGINT NOT NULL REFERENCES entity(id),
  rel         TEXT NOT NULL,                       -- director_of|shares_address|co_party|family_of|c
  confidence   NUMERIC(3,2) NOT NULL,
  fetch_id    BIGINT REFERENCES src_fetch(id),
  valid_from   DATE,
  valid_to     DATE,
  UNIQUE (from_id, to_id, rel)
);
CREATE INDEX ON entity_link (from_id, rel);
CREATE INDEX ON entity_link (to_id, rel);

```

6.4 Entities

6.5 The fact store This is the core design decision. **Never overwrite a fact. Add a competing one.**

```
CREATE TABLE fact (
  id          BIGSERIAL PRIMARY KEY,
  subject_type TEXT NOT NULL,           -- 'parcel'|'entity'|'project'
  subject_id  BIGINT NOT NULL,
  predicate   TEXT NOT NULL,           -- 'has_owner'|'area_ha'|'land_class'|...
  object_entity BIGINT REFERENCES entity(id),
  object_value TEXT,                   -- for scalar facts
  object_num   NUMERIC,                -- for numeric facts (indexed separately)
  valid_from   DATE,
  valid_to     DATE,                   -- NULL = currently believed
  confidence   NUMERIC(3,2) NOT NULL,
  fetch_id     BIGINT NOT NULL REFERENCES src_fetch(id),
  superseded_by BIGINT REFERENCES fact(id),
  recorded_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX ON fact (subject_type, subject_id, predicate);
CREATE INDEX ON fact (predicate, object_num) WHERE object_num IS NOT NULL;
CREATE INDEX ON fact (object_entity) WHERE object_entity IS NOT NULL;
CREATE INDEX ON fact (subject_type, subject_id, predicate)
  WHERE valid_to IS NULL AND superseded_by IS NULL;
```

Current-belief view — this is what the API reads, never the raw table:

```
CREATE VIEW fact_current AS
SELECT DISTINCT ON (subject_type, subject_id, predicate)
  *
FROM fact
WHERE superseded_by IS NULL AND valid_to IS NULL
ORDER BY subject_type, subject_id, predicate, confidence DESC, recorded_at DESC;
```

Predicate catalogue — a config table, so adding a government field is a row insert, not a migration:

```
CREATE TABLE predicate_catalogue (
  predicate TEXT PRIMARY KEY,
  label     TEXT NOT NULL,
  value_type TEXT NOT NULL,           -- 'entity'|'text'|'numeric'|'date'
  unit      TEXT,
  decay_pct_yr NUMERIC(4,2) NOT NULL DEFAULT 5, -- confidence decay per year
  notes     TEXT
);
```

```
CREATE TABLE deed (
  id          BIGSERIAL PRIMARY KEY,
  sro_office  TEXT NOT NULL,
  doc_no      TEXT NOT NULL,
  doc_year    INT NOT NULL,
  doc_type    TEXT NOT NULL,           -- sale|gift|mortgage|lease|gpa|spa|ats|partition
  exec_date   DATE,
  reg_date    DATE,
  consideration NUMERIC(16,2),         -- REGISTERED value. Understated. Never call it ma
  stamp_duty   NUMERIC(14,2),
  property_text TEXT NOT NULL,         -- raw free-text description
```

```

parcel_id      BIGINT REFERENCES parcel(id),
parcel_match   TEXT NOT NULL DEFAULT 'unmatched', -- exact|fuzzy|unmatched
fetch_id      BIGINT NOT NULL REFERENCES src_fetch(id),
UNIQUE (sro_office, doc_no, doc_year)
);
CREATE INDEX ON deed (parcel_id, reg_date DESC);
CREATE INDEX ON deed (doc_type, reg_date DESC);
CREATE INDEX ON deed (parcel_match) WHERE parcel_match = 'unmatched';

CREATE TABLE deed_party (
  id            BIGSERIAL PRIMARY KEY,
  deed_id      BIGINT NOT NULL REFERENCES deed(id),
  entity_id    BIGINT REFERENCES entity(id),
  role         TEXT NOT NULL,                -- seller|buyer|witness|attorney|guardian
  name_raw     TEXT NOT NULL,                -- always keep the raw string
  match_conf   NUMERIC(3,2)
);
CREATE INDEX ON deed_party (deed_id);
CREATE INDEX ON deed_party (entity_id, role);

```

6.6 Transactions

```

CREATE TABLE rera_project (
  id            BIGSERIAL PRIMARY KEY,
  rera_id      TEXT UNIQUE NOT NULL,
  name         TEXT,
  promoter_entity BIGINT REFERENCES entity(id),
  district_id  BIGINT REFERENCES admin_unit(id),
  units_total  INT,
  declared_completion DATE,
  status       TEXT,
  extensions   INT NOT NULL DEFAULT 0,
  geom         geometry(Point, 4326),
  fetch_id     BIGINT REFERENCES src_fetch(id)
);
CREATE INDEX ON rera_project USING GIST (geom);

CREATE TABLE auction_listing (
  id            BIGSERIAL PRIMARY KEY,
  source        TEXT NOT NULL,                -- ibapi|drt|mp_eauction
  ext_ref       TEXT NOT NULL,
  lender        TEXT,
  borrower_text TEXT,
  property_text TEXT NOT NULL,
  reserve_price NUMERIC(16,2),
  emd           NUMERIC(14,2),
  auction_date  DATE,
  possession    TEXT,                        -- symbolic|physical|unknown ← SURFACE THIS
  attempt_no    INT DEFAULT 1,
  parcel_id     BIGINT REFERENCES parcel(id),
  model_value   NUMERIC(16,2),
  discount_pct  NUMERIC(5,2),
  first_seen    TIMESTAMPTZ NOT NULL DEFAULT now(),
  fetch_id     BIGINT REFERENCES src_fetch(id),

```

```

    UNIQUE (source, ext_ref)
);
CREATE INDEX ON auction_listing (auction_date, discount_pct DESC);

CREATE TABLE infra_project (
    id                BIGSERIAL PRIMARY KEY,
    source            TEXT NOT NULL,
    ext_ref           TEXT,
    title             TEXT NOT NULL,
    agency            TEXT,
    sector            TEXT,
    value_cr          NUMERIC(12,2),
    stage             TEXT NOT NULL,          -- nit|awarded|work_order|in_progress|complete|sta
    stage_date        DATE,
    geom              geometry(Geometry, 4326), -- point or corridor buffer
    geocode_conf      NUMERIC(3,2),
    fetch_id          BIGINT REFERENCES src_fetch(id)
);
CREATE INDEX ON infra_project USING GIST (geom);

CREATE TABLE zone (
    id                BIGSERIAL PRIMARY KEY,
    plan_name         TEXT NOT NULL,          -- 'Indore Master Plan 2035'
    zone_code         TEXT NOT NULL,
    zone_label        TEXT NOT NULL,
    permissible_fsi   NUMERIC(4,2),
    is_no_dev         BOOLEAN NOT NULL DEFAULT false,
    geom              geometry(MultiPolygon, 4326) NOT NULL,
    georef_rms_m      NUMERIC(6,2),          -- georeferencing error. Flag if > 15.
    source_doc        TEXT
);
CREATE INDEX ON zone USING GIST (geom);

CREATE TABLE circle_rate (
    id                BIGSERIAL PRIMARY KEY,
    district_id       BIGINT REFERENCES admin_unit(id),
    location_text     TEXT NOT NULL,
    land_class        TEXT,
    rate_per_sqm      NUMERIC(14,2) NOT NULL,
    effective_from    DATE NOT NULL,
    effective_to      DATE,
    geom              geometry(MultiPolygon, 4326)
);

```

6.7 Projects, auctions, infrastructure, zones

```

CREATE TABLE metric_value (
    id                BIGSERIAL PRIMARY KEY,
    scope_type        TEXT NOT NULL,          -- 'parcel' | 'h3_r8' | 'h3_r7' | 'village'
    scope_key          TEXT NOT NULL,
    metric            TEXT NOT NULL,          -- 'txn_velocity' | 'price_trend' | 'nightlight_growth'
    period            DATE NOT NULL,          -- period start
    value             NUMERIC NOT NULL,
    sample_n          INT,                   -- how many observations. NULL-safe: low n = low t

```

```

    computed_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
    calc_version   TEXT NOT NULL,
    UNIQUE (scope_type, scope_key, metric, period, calc_version)
);
CREATE INDEX ON metric_value (metric, period DESC, value DESC);
CREATE INDEX ON metric_value (scope_type, scope_key, metric);

```

6.8 Derived metrics The 12 v1 metrics. Build only these:

Metric	Scope	Definition
txn_velocity	h3_r8, quarterly	deeds ÷ parcels in cell
price_trend	h3_r8, quarterly	OLS slope of median ₹/sqm over 8 quarters
price_median	h3_r8, quarterly	median registered ₹/sqm
below_circle_share	h3_r8, quarterly	share of deeds under benchmark
owner_concentration	h3_r8	Herfindahl over resolved clusters
new_entrant_share	h3_r8, quarterly	buyers never before seen in district
flip_share	h3_r8, annual	resales within 18 months
nightlight_growth	h3_r7, annual	VIIRS YoY radiance change
builtup_change	h3_r8, annual	NDBI-derived built fraction delta
poa_density	h3_r8, quarterly	POA + ATS registrations ÷ parcels
dist_arterial_m	parcel	distance to nearest classified road
fsi_headroom	parcel	permissible FSI – estimated built FSI (Indore only)

```

CREATE TABLE scraper_run (
  id          BIGSERIAL PRIMARY KEY,
  source      TEXT NOT NULL,
  started_at  TIMESTAMPTZ NOT NULL,
  ended_at    TIMESTAMPTZ,
  status      TEXT NOT NULL,           -- running|ok|failed|blocked
  rows_fetched INT DEFAULT 0,
  rows_parsed INT DEFAULT 0,
  rows_failed INT DEFAULT 0,
  cursor      JSONB,                  -- resume point
  error       TEXT
);
CREATE INDEX ON scraper_run (source, started_at DESC);

-- Anything the machine could not decide. A human clears this queue.
CREATE TABLE review_queue (
  id          BIGSERIAL PRIMARY KEY,
  kind        TEXT NOT NULL,           -- entity_merge|parcel_match|parse_fail|geom_invalid
  payload     JSONB NOT NULL,
  confidence   NUMERIC(3,2),
  status      TEXT NOT NULL DEFAULT 'open', -- open|resolved|rejected
  resolved_by TEXT,
  resolved_at  TIMESTAMPTZ,
  created_at   TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX ON review_queue (kind, status, confidence);

CREATE TABLE watchlist (
  id          BIGSERIAL PRIMARY KEY,
  user_email  TEXT NOT NULL,

```

```

subject_type TEXT NOT NULL,
subject_id BIGINT NOT NULL,
note TEXT,
created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
UNIQUE (user_email, subject_type, subject_id)
);

CREATE TABLE alert_event (
  id BIGSERIAL PRIMARY KEY,
  watchlist_id BIGINT REFERENCES watchlist(id),
  kind TEXT NOT NULL,
  detail JSONB NOT NULL,
  sent_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Log every prediction so you eventually learn which screens work.
CREATE TABLE prediction_log (
  id BIGSERIAL PRIMARY KEY,
  screen_id TEXT NOT NULL,
  parcel_id BIGINT REFERENCES parcel(id),
  predicted JSONB NOT NULL,
  predicted_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  outcome JSONB,
  outcome_at TIMESTAMPTZ
);

```

6.9 Operations

7. Pipelines

Scheduling (systemd timers)

Job	Cadence	Owner
fetch_auctions	every 6h	Dev A
fetch_sampada_indore	daily 02:00	Dev A
fetch_sampada_other	weekly Sun	Dev A
fetch_rera	weekly Mon	Dev A
fetch_tenders	weekly Mon	Dev A
fetch_bhulekh_sweep	continuous, resumable	Dev A
canonicalise	hourly	Dev A
entity_resolve	nightly 03:00	Dev A
compute_metrics	nightly 04:00	Dev A
refresh_tiles	nightly 05:00	Dev B
run_alerts	daily 07:00	Dev B
fetch_satellite	quarterly, manual trigger	Dev A

Canonicalisation rules

Skip any `src_fetch` whose `content_hash` matches the previous fetch of the same document. Parse → staging → canonical. Never write canonical tables directly from a scraper. Every parse failure writes to `review_queue` with the raw payload key. Never silently drop. New facts are inserted. Contradicting facts get superseded_by set on the old row — the old row stays. Every canonical row carries a `fetch_id`. No exceptions. A row without provenance is a bug.

Entity resolution — deterministic first

Run in this order, stop at first match:

Exact CIN → same company. Confidence 1.00. Exact (name, father_name, village) → same person. Confidence 0.95. Fuzzy: pg_trgm similarity on name ≥ 0.85 AND same village AND father_name similarity ≥ 0.8 → confidence 0.75, write to review_queue, do not auto-merge. Anything below → new entity.

Hard rules:

Auto-merge only at confidence ≥ 0.90 . Merges are reversible: keep both entity rows, link via cluster_id. Never delete an entity. Any screen result that depends on a cluster with confidence < 0.85 must show a warning badge in the UI. Do not build an ML entity resolver in v1. The deterministic rules plus a review queue will get you to ~90% and take one week instead of three months.

8. API

FastAPI. Twelve endpoints. That is the whole surface.

GET /api/tiles/parcels/{z}/{x}/{y}.pbf	vector tiles, ST_AsMVT
GET /api/tiles/{layer}/{z}/{x}/{y}.pbf	zones, projects, auctions, metrics
GET /api/parcel/{id}	full dossier
GET /api/parcel/{id}/history	deed + fact timeline
GET /api/parcel/{id}/comparables	nearby deeds, 2km / 18mo
GET /api/entity/{id}	entity + all holdings
GET /api/entity/{id}/network	1–2 hop graph (recursive CTE)
GET /api/search?q=	khasra / owner / village / project
POST /api/screen/run	{screen_id, params, bbox} → results
GET /api/screens	list of 30 screen definitions
POST /api/watchlist	add/remove
GET /api/auctions/live	ranked by discount_pct

Tile strategy

Serve from ST_AsMVT directly. Cache in R2, front with Cloudflare. Regenerate nightly.

Zoom rules — non-negotiable for performance:

Zoom	Layer served
6–9	district polygons with a metric fill
10–12	h3_r7 cells
13–14	village polygons / h3_r8 cells
15–16	parcel centroids, clustered
17+	full parcel polygons + khasra labels

Do not serve parcel geometry below z15. This is the single most important performance decision in the frontend.

Screen definitions

Each of the 30 screens is a JSON file in /api/screens/*.json:

```
{
  "id": "poa_cluster",
  "label": "Power of Attorney cluster",
  "description": "Multiple POAs registered to the same holder across different owners in one",
  "params": [
    { "key": "min_grantors", "type": "int", "default": 3, "min": 2, "max": 10 },
```

```

    {"key": "radius_m", "type": "int", "default": 1000},
    {"key": "months", "type": "int", "default": 24}
  ],
  "sql_template": "screens/poa_cluster.sql",
  "caveat": "Requires the full SRO document index, not just sale deeds."
}

```

Rule: SQL lives in a versioned file. Parameters are bound, never string-interpolated. Every screen result must display its caveat text alongside the results — not in a tooltip, in the result header.

9. Frontend

Five screens. Nothing else.

9.1 Map (the default view) MapLibre, full viewport Layer panel: 6 layers in v1 — parcels, zoning, transaction velocity, price trend, RERA projects, auctions Each layer: on/off, opacity Time slider for the temporal metrics (quarterly steps) Click a parcel → dossier panel slides in from the right Draw-polygon tool → sets the spatial scope for a screen run

Parcel styling by boundary_tier — mandatory:

Tier	Style
A	solid outline, filled
B	dashed outline, hatched fill, tooltip “approximate location”
C	not rendered on the map; reachable only via search

9.2 Parcel dossier Sections, in this order:

Identity — khasra, ULPIN, village, area (recorded vs geometry, flagged if they differ >10%), boundary tier **Ownership** — current owners, resolved cluster (with confidence badge), co-owner count, cultivator entries **History** — every deed, chronological, with the mutation-lag flag per transfer **Valuation** — comparables table, circle rate, registered ₹/sqm with the understatement warning **Regulatory** — zone, permissible FSI, no-dev flags (Indore only) **Risk flags** — every title check that fired, with severity **Owner’s other holdings** — parcels in the same cluster **Provenance** — collapsible: every fact, its source, fetch date, confidence, link to raw document

9.3 Screen runner Left: list of 30 screens, grouped by domain Middle: parameter form, auto-generated from the screen JSON Right: results as a sortable table + a map layer Above results: the screen’s caveat text, always visible Actions: export CSV, add all to watchlist, save this configuration

9.4 Auction board Live MP auction listings, default sort by discount_pct descending Columns: property, lender, reserve, our model value, discount %, possession type, auction date, days remaining Possession = symbolic renders with a warning colour and a tooltip explaining the eviction risk Click → parcel dossier if matched, raw notice if not

9.5 Review queue The human-in-loop UI. Entity merges, parcel matches, parse failures. Two buttons: confirm, reject. Keyboard-driven. This screen is not optional. Data quality decays without it.

10. Precautions

These override everything else in this document.

Scraping conduct

Rate limits are hard limits. 1 request per 2-3 seconds per source per IP. Never faster, even if the portal tolerates it. Never defeat a block. If a portal blocks you, stop for 24 hours and reduce your rate. Do not rotate IPs to circumvent. Use a real, identifiable User-Agent with a contact email. Free/public endpoints only. Do not scrape behind login or paid-certified-copy paths. Every scraper must be resumable from a persisted cursor. Blocked or failing scrapers alert immediately — a silently broken scraper is worse than a loudly broken one.

Data integrity

Never overwrite a fact. Contradictions are added, with the older row marked superseded. Every canonical row has a `fetch_id`. A row without provenance does not ship. Never present registered consideration as market value. Anywhere in the UI, it is labelled “registered value (understated)”. Never render an approximated boundary as surveyed. Tier B/C styling is mandatory. Never display “no litigation” or “no encumbrance”. Display “none found in {source} as of {date}”. Every metric with `sample_n` below 5 displays a low-confidence indicator. Reject any geometry outside the MP bounding box. Log it, do not store it. Auto-merge entities only at confidence ≥ 0.90 . Everything else goes to review.

Legal and ethical

Public sources only. If it needs a login or a payment to access, it is out of scope for v1. No person-level political or official connectivity analysis in v1. It is deferred deliberately — build it only after counsel has reviewed the DPDP position. Access is logged. Every screen run and dossier view writes a row with user and timestamp. Do not act on any information that is not genuinely public. This constrains what the system is for, not just what it stores.

Engineering

Migrations only. No manual DDL on the server, ever. Every scraper has a fixture test with a saved sample response. CI runs it. Raw payloads are never deleted. R2 storage is cheap; re-scraping is not. `parser_version` on every fetch, so you can re-parse the raw lake when the parser improves. Nightly `pg_dump` to R2, retained 30 days. Test the restore once, in week 4.

11. KPIs

Reviewed every Friday in the demo. If a red number persists two weeks, that becomes the week’s priority.

Coverage

KPI	Week 8 target	Week 16 target
Indore parcels ingested	60%	98%
MP parcels ingested (all districts)	10%	70%
Indore parcels with tier-A geometry	30%	55%
Indore deeds ingested (2015→)	70%	98%
Deeds matched to a parcel	65%	85%
Indore zone coverage	50%	100%

Freshness

KPI	Target
Auction listing → in system	< 6 hours

KPI	Target
Auction listing → scored + alerted	< 12 hours
SAMPADA deed → in system (Indore)	< 48 hours
Bhulekh full Indore sweep	< 30 days

Note: monitor digitized regional newspapers for land acquisition announcements as they are published — these are often the earliest public record of an acquisition, and can serve as dated evidence in future court proceedings, ahead of the acquisition appearing in any government portal.

Quality

KPI	Target
Scraper success rate (rolling 7d)	> 95%
Parse failure rate	< 3%
Entity resolution precision (100-sample manual audit, monthly)	> 90%
Review queue depth	< 500 open, none older than 14 days
Facts without provenance	0

Platform

KPI	Target
Tile response (p95)	< 300 ms
Parcel dossier load (p95)	< 800 ms
Screen run, district scope (p95)	< 5 s
Uptime	> 99%

The ones that actually matter

KPI	Week 16 target
Auctions auto-screened	> 200
Opportunities the team evaluated using the system	> 20
Bids placed	≥ 3
Predictions logged in prediction_log	> 100

The last four are the only KPIs that tell you whether the system is worth anything.

12. Milestones

Weeks 1-2 — Foundations

Both: schema agreed and migrated. Repo, CI, VM, Postgres+PostGIS, R2. Manual walkthrough of all 12 portals, fixtures saved.

Dev A: src_fetch + scraper_run plumbing. Bhulekh scraper working for one tehsil. Dev B: FastAPI skeleton, auth, admin_unit loaded, MP boundaries rendering on a MapLibre map.

Ships: a map of MP with district boundaries and one tehsil of khasra records in the database.

Weeks 3-4 — Land records

Dev A: Bhulekh across all Indore tehsils. Bhu-Naksha geometry ingestion with tier classification. Dev B: parcel vector tiles with zoom strategy. Basic dossier: identity + ownership. Search.

Ships: zoom into Indore, see real khasra parcels, click one, see who owns it.

Weeks 5-6 — Transactions

Dev A: SAMPADA scraper — full document index including POA/ATS. Deed→parcel matching. Circle rates. Dev B: dossier history section. Comparables. Deed timeline UI.

Ships: a parcel's complete transaction history, with comparables.

Weeks 7-8 — Auctions and entities

Dev A: auction feeds (all 3). Entity resolution v1 + review queue backend. MCA on-demand. Dev B: auction board. Review queue UI. Entity page with holdings.

Ships: the auction board, scoring live MP listings. **First real bid should be possible here.**

Weeks 9-10 — Metrics and layers

Dev A: all 12 metrics computing nightly. VIIRS statewide. Sentinel for Indore. Dev B: metric layers on the map, time slider, layer panel.

Ships: the full six-layer map with temporal scrubbing.

Weeks 11-12 — Screening

Dev A: RERA, tenders, zoning ingestion for Indore. Title-check rules. Dev B: screen runner UI, 30 screen definitions wired, CSV export.

Ships: the screening tool, running all 30 screens against real data.

Weeks 13-14 — Statewide and alerts

Dev A: Bhulekh + SAMPADA sweep across remaining 54 districts at ownership+deed depth. Dev B: watchlist, alert engine, daily digest email.

Ships: statewide coverage, alerting live.

Weeks 15-16 — Harden

Both: backfill gaps, clear the review queue, tune performance, fix the top 20 data quality issues, write the runbook.

Ships: v1.

13. Acceptance criteria

v1 is done when every one of these passes:

- ☐ 98% of Indore khasra records present, with a coverage report by tehsil
- ☐ 85% of Indore deeds matched to a parcel; the unmatched 15% enumerated in review_queue
- ☐ Every parcel renders at z17+ with correct boundary-tier styling
- ☐ Parcel dossier shows all 8 sections, with a working provenance trail to the raw document
- ☐ All 30 screens run against Indore and return results in under 5 seconds
- ☐ Every screen displays its caveat text in the result header
- ☐ Auction board shows live MP listings, ranked by discount, with possession type visible
- ☐ A new MP land auction triggers an alert within 12 hours of appearing
- ☐ Entity resolution audit on a 100-row manual sample shows > 90% precision
- ☐ Review queue has fewer than 500 open items, none older than 14 days
- ☐ Zero canonical rows without a fetch_id
- ☐ Database restore from backup tested and documented
- ☐ Runbook exists: how to restart each scraper, what each alert means, how to clear the review queue
- ☐ The team has evaluated 20+ opportunities and placed at least 3 bids using the system

14. What to do when you disagree with this document

Bring it up in the Friday demo. Two rules:

If you want to **add** something, say what you will cut to pay for it. If you want to **cut** something, say which acceptance criterion it breaks.

The most likely failure mode for this project is not a technical one. It is building a wider system than two people can maintain, and arriving at week 16 with impressive coverage and zero decisions made. Indore, deeply, beats MP shallowly — every week.